EE2C2 Mixed-Signal Circuits and Systems
Lab 3: Digital Circuits on FPGA

Chang Gao*

10 June 2026

How to use this handout

GUI	Click this path of menus/buttons in Vivado. Breadcrumbs are written **A** → **B** → **C**.
PowerShell	Type this in a Windows PowerShell window (dark code block).
Tcl Console	Type this in the **Tcl Console** at the bottom of the Vivado window (green code block).

Checkpoint (yellow box): a short question; write your answer in the provided report template (see the note below).
Note (orange box): a pitfall or important detail.

Note

- **The GUI and the command line do the same thing.** Vivado is scriptable: every time you click a button, it prints the equivalent **Tcl** command in the Tcl Console. Watching that console is the fastest way to learn the commands below. PowerShell simply runs Vivado in batch mode (`-mode batch`), i.e. it runs a Tcl script with no window.
- **Answer the checkpoints in the report template.** Each **Checkpoint** question must be answered in the provided **LaTeX report template** (`EE2C2_Lab3_Report_Template`) on **Brightspace**; submit the compiled report PDF there. Do not write your answers in this handout.
- **AUP-ZU3 box contents and return.** The AUP-ZU3 box contains one FPGA board, one USB-C-to-USB-C cable for power, one USB-A-to-USB-C cable for the PC connection, and one power adapter. Please keep all items together and return the complete box after the lab.

*Email: chang.gao@tudelft.nl

1 Topic 0 — Getting started: Vivado, the design flow, and the project

1.1 Background

A **Field-Programmable Gate Array (FPGA)** is a chip full of configurable look-up tables, flip-flops, wiring, and I/O. You describe hardware in **SystemVerilog**; the tool configures the FPGA to *become* that hardware. **Vivado** is the design tool. The path from source to a running board is the **design flow**:

```
SystemVerilog (RTL) --simulate--> Synthesis (RTL -> LUT/FF netlist)
   --> Implementation (place & route) --> Bitstream (.bit) --> Program over JTAG
```

You will walk the tool flow in Topics 1–6 using an **AC701 / Artix-7** project, because that board and device support are installed on the university PCs by default. The physical lab board on the bench is the **RealDigital AUP-ZU3-4GB**; the AUP-ZU3 demos in Topics 7–8 are programmed from **prebuilt bitstreams** so you can still see the LED behaviour even if the campus Vivado installation does not include XCZU3EG synthesis/implementation support.

1.2 Lab 0: clone the repo, create the project, and tour Vivado

Step 0.1 — Clone the lab repository and open it in PowerShell.

The lab lives on GitHub at https://github.com/lab-emi/EE2C2_FPGA_Lab.

PowerShell open Windows PowerShell, go to your home folder with `cd ~` (PowerShell expands `~` to `C:\Users\<you>`), then clone and enter it:

```
cd ~                    # your user home folder, e.g. C:\Users\<you>
git clone https://github.com/lab-emi/EE2C2_FPGA_Lab.git
cd EE2C2_FPGA_Lab
dir
```

You should now see `build.tcl`, `prebuilt_bitstreams`, and the `src`, `sim`, `constr`, `scripts` folders (now under `C:\Users\<you>\EE2C2_FPGA_Lab`).

> **Note**
>
> **No Git?** On the GitHub page choose **Code → Download ZIP** and extract it; the rest of the lab is identical once you `cd` into the extracted folder.

Add Vivado to the PATH for this PowerShell window, so you can type `vivado` instead of its full path:

```
$env:Path += ";C:\Programs\Xilinx2023.2\Vivado\2023.2\bin"
$env:Path += ";C:\Xilinx\Vivado\2023.2\bin"
vivado -version
```

`vivado -version` should print `Vivado v2023.2`. From now on this handout writes `vivado ...` instead of the full path, and assumes your prompt is **inside the cloned `EE2C2_FPGA_Lab` folder**.

> **Note**
>
> **Still says `vivado` is not recognised?** Check whether Vivado is installed in a different folder, then replace the path above with the correct `bin` folder.

```
PS C:\WINDOWS\system32> cd ~
PS C:\Users\cgao> git clone https://github.com/lab-emi/EE2C2_FPGA_Lab.git
Cloning into 'EE2C2_FPGA_Lab'...
remote: Enumerating objects: 91, done.
remote: Counting objects: 100% (91/91), done.
remote: Compressing objects: 100% (64/64), done.
remote: Total 91 (delta 17), reused 86 (delta 15), pack-reused 0 (from 0)
Receiving objects: 100% (91/91), 3.62 MiB | 14.44 MiB/s, done.
Resolving deltas: 100% (17/17), done.
PS C:\Users\cgao> cd .\EE2C2_FPGA_Lab\
PS C:\Users\cgao\EE2C2_FPGA_Lab> dir


    Directory: C:\Users\cgao\EE2C2_FPGA_Lab


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         2026/5/31     18:50                board-files
d-----         2026/5/31     18:50                constr
d-----         2026/5/31     18:50                constraints
d-----         2026/5/31     18:50                lab_handout
d-----         2026/5/31     18:50                legacy_rainbow_src
d-----         2026/5/31     18:50                optional_pwm_dac
d-----         2026/5/31     18:50                scripts
d-----         2026/5/31     18:50                sim
d-----         2026/5/31     18:50                src
-a----         2026/5/31     18:50           1849 .gitignore
-a----         2026/5/31     18:50           3104 build.tcl
-a----         2026/5/31     18:50           1121 LICENSE
-a----         2026/5/31     18:50           1742 program_jtag.tcl
-a----         2026/5/31     18:50           4464 README.md


PS C:\Users\cgao\EE2C2_FPGA_Lab> $env:Path += ";C:\Xilinx\Vivado\2023.2\bin"
PS C:\Users\cgao\EE2C2_FPGA_Lab> vivado -version
ECHO is off.
ECHO is off.
vivado v2023.2 (64-bit)
Tool Version Limit: 2023.10
SW Build 4029153 on Fri Oct 13 20:14:34 MDT 2023
IP Build 4028589 on Sat Oct 14 00:45:43 MDT 2023
SharedData Build 4025554 on Tue Oct 10 17:18:54 MDT 2023
Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
Copyright 2022-2023 Advanced Micro Devices, Inc. All Rights Reserved.
```

Figure 1: PowerShell ready in the cloned lab folder.

Step 0.2 — Create the Vivado project.

PowerShell this builds the project without synthesis (fast):

```
vivado -mode batch -source scripts\create_vivado_project.tcl
```

Watch for `INFO: Using AC701 implementation part: xc7a200tfbg676-2` and `INFO: Created project: ...\build\EE2C2_FPGA_Lab.xpr`. This project targets the AC701 board part `xilinx.com:ac701:part0` it is the project you use for simulation setup, synthesis, implementation, timing, and power reports.

Note

`create_vivado_project.tcl` deletes and recreates the `build\` folder every run. That is fine, because all sources live in `src\`, `sim\`, `constr\`, never in `build\`.

```
INFO: Created project: C:/Users/cgao/EE2C2_FPGA_Lab/build/EE2C2_FPGA_Lab.xpr
INFO: [Common 17-206] Exiting Vivado at Sun May 31 19:34:26 2026...
PS C:\Users\cgao\EE2C2_FPGA_Lab>
```

Figure 2: Project created from the Tcl script.

Step 0.3 — Open the project in the GUI.

PowerShell

```
vivado build\EE2C2_FPGA_Lab.xpr
```

GUI *(alternative)* Start menu → **Vivado 2023.2** → **Open Project** → select `build\EE2C2_FPGA_Lab.xpr`.

Find the four areas you will use constantly: **Flow Navigator** (left), **Sources** (centre), **Tcl Console** (bottom), and **Messages** (bottom). Try typing `version -short` in the Tcl Console.

Step 0.4 — Find the source files.

GUI In **Sources**, expand **Design Sources** (`top`, `comb_led_logic`, `one_hz_tick`, `led_fsm`), **Constraints** (`ac701_lab.xdc`), and **Simulation Sources** (`tb_comb_led_logic`, `tb_led_fsm`).

Tcl Console *(alternative)*

```
report_compile_order -fileset sources_1
get_files
```

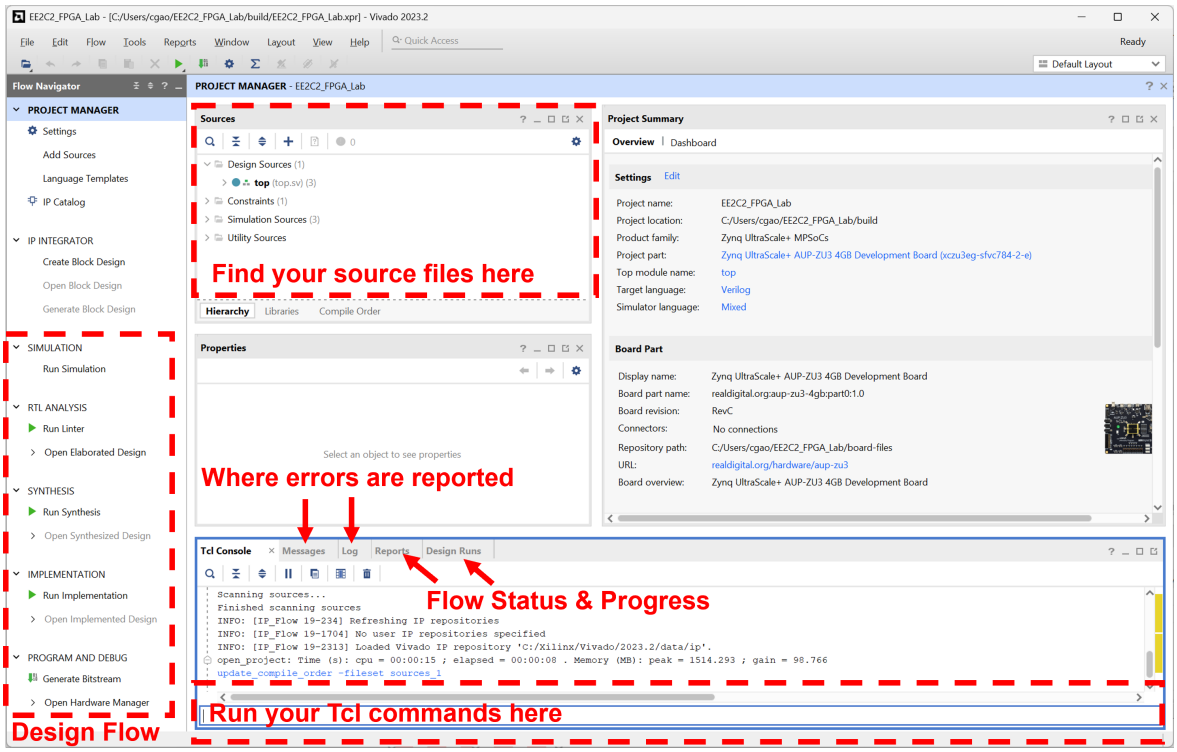

Figure 3: The Vivado main window.

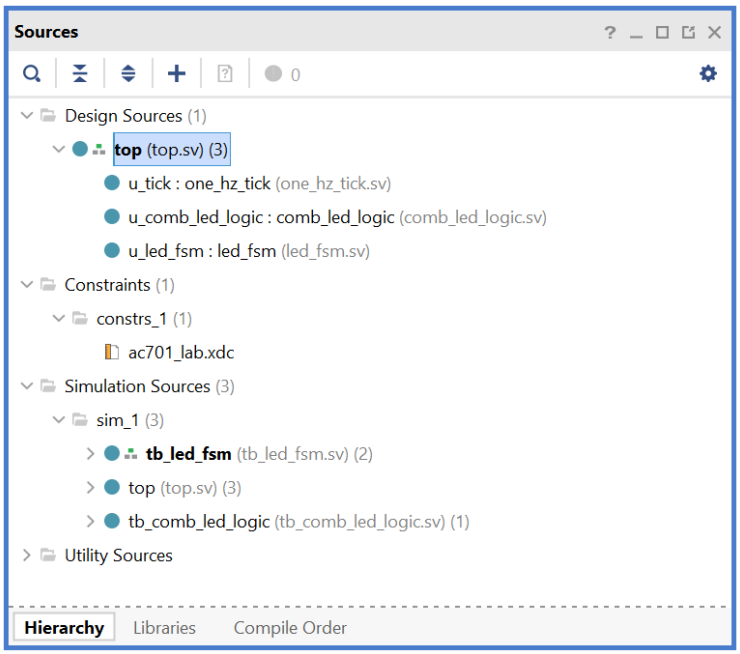

Figure 4: The Sources panel, fully expanded.

2 Topic 1 — Combinational logic

2.1 Background

Combinational logic produces outputs that depend *only on the current inputs*; it has no memory of the past. Gates, multiplexers, decoders, and comparators are all combinational. Because there is no clock and no stored state, the output is not "computed once" like a line of software; it is a piece of hardware that is *always present* and continuously reflects its inputs. When an input changes, the output settles to its new value after a short **propagation delay** through the gates.

In SystemVerilog you describe combinational logic either with a continuous `assign` or inside an `always_comb` block, and both build the same kind of hardware:

```
assign y = (a & b) | (~a & c);   // continuous assignment

always_comb begin                // procedural form, still combinational
    y = (a & b) | (~a & c);
end
```

The lab module `src/comb_led_logic.sv` is a single `always_comb` block that drives eight LED bits, each a different function of the four switches `sw[3:0]` and the button `btn`. Its SystemVerilog operators (`&`, `|`, `^`, `~`, the `? :` conditional, and the reduction `&sw`) each map directly onto a piece of logic hardware. Working out which is which is the goal of Checkpoint 1a.

Note

Avoid latches. Inside an `always_comb` block, every output must be assigned on every path through the block. If a bit is left unassigned for some input combination, the synthesiser has to "remember" its previous value and infers an unwanted memory element, a **latch**. The lab code prevents this by writing a default `led_comb = 8'b0000_0000;` as its first line, so every bit always has a defined value.

2.2 Lab 1: inspect and simulate the combinational logic

Step 1.1 — Open and read the source.

GUI **Sources** → **Design Sources** → double-click `comb_led_logic`. The whole module is one `always_comb` block. Read it top to bottom: the first line clears all eight outputs (`led_comb = 8'b0000_0000;`), and then each `led_comb[i]` is assigned a different combinational function of `sw` and `btn`. Before you simulate, work out what gate or function each line builds, and watch for the two lines that both use `&` but mean different things.

Checkpoint: answer in the report template

Checkpoint 1a: reading the logic.

1. Identify the line that implements an **AND**, an **OR**, a **NOT**, an **XOR**, and a **multiplexer**.
2. `led_comb[1] = sw[0] & sw[1]` and `led_comb[7] = &sw` both use `&`, yet they build different gates. Explain the difference, and state exactly which switch settings drive each of these two outputs high.

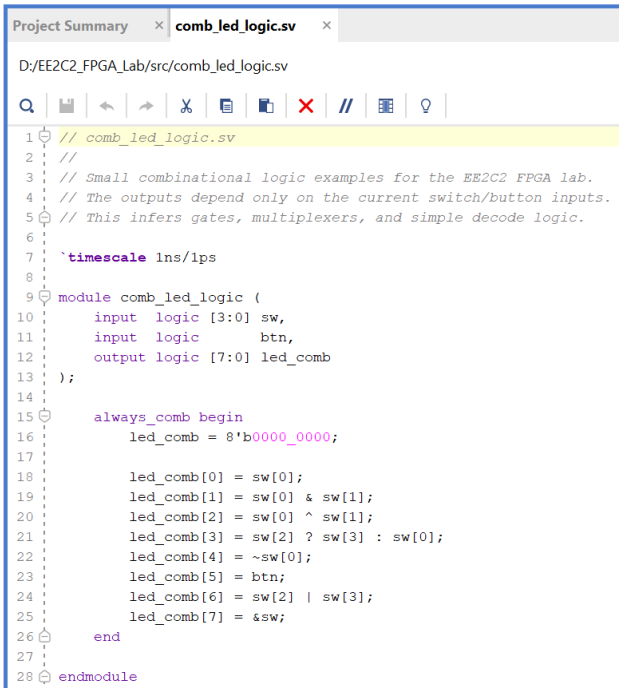

Figure 5: `comb_led_logic.sv` in the editor.

Step 1.2 — Make the combinational testbench the simulation top.

GUI **Sources → Simulation Sources → sim_1 →** right-click `tb_comb_led_logic` **→ Set as Top**.

Tcl Console *(alternative)*

```
set_property top tb_comb_led_logic [get_filesets sim_1]
update_compile_order -fileset sim_1
```

Step 1.3 — Run the behavioural simulation.

GUI **Flow Navigator → Run Simulation → Run Behavioral Simulation**.

Note

Close the previous simulation before launching another one. If a waveform/simulator window is still open, run `close_sim` in the Tcl Console (or close the current simulation from the GUI) before the next `launch_simulation`. Otherwise Vivado may report `ERROR: [Vivado 12-1501] Simulator for snapshot ... is already running`.

Tcl Console *(alternative)*

```
catch {close_sim} ;# harmless if no simulation is open
launch_simulation
run all
```

PowerShell *(alternative, no GUI)*

```
vivado -mode batch -source scripts\run_simulation.tcl -tclargs tb_comb_led_logic
```

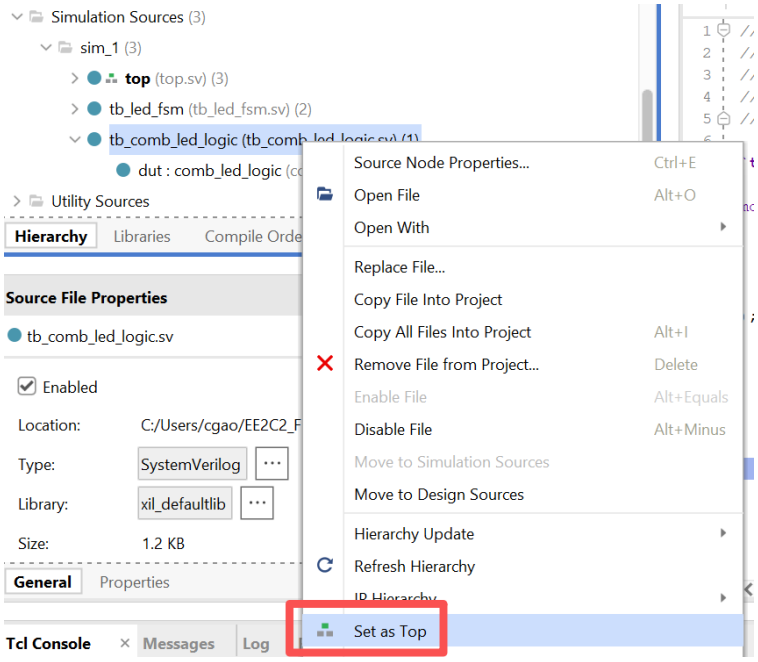

Figure 6: Setting the simulation top.

The Tcl console should print lines like `sw=1111 btn=0 led_comb=...` ending with `tb_comb_led_logic completed.`

Step 1.4 — Read the waveform.

GUI Make sure `sw`, `btn`, and `led_comb` are in the wave window (drag in any that are missing) and click **Zoom Fit**. Expand `led_comb` into its individual bits so you can follow each function separately. Notice that every time `sw` or `btn` changes, the affected `led_comb` bits change at the *same* simulation time; there is no clock edge to wait for, because combinational logic has no memory.

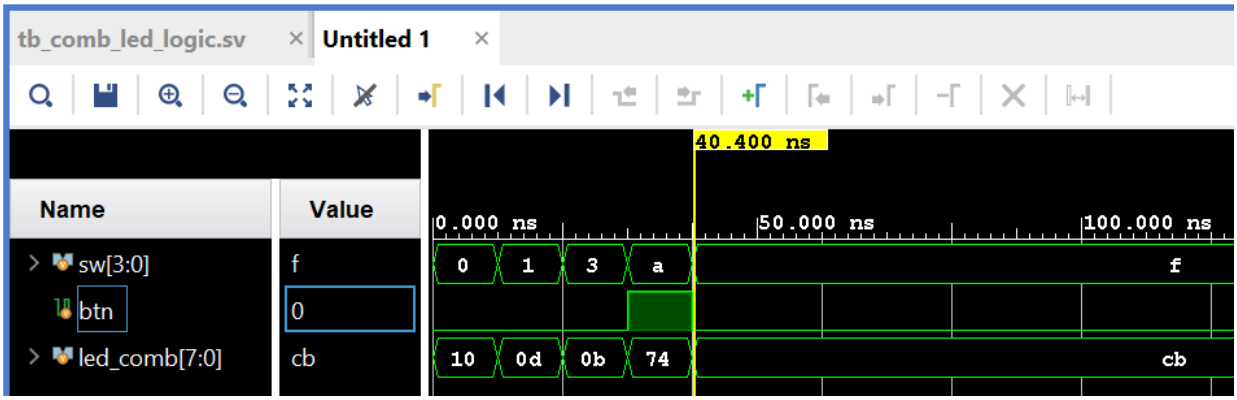

Figure 7: Behavioural-simulation waveform for the combinational logic.

Checkpoint: answer in the report template

Checkpoint 1b: combinational behaviour and simulation.

1. In the waveform, when `sw` changes, do the outputs change on a clock edge or immediately? Why?
2. Why may a *testbench* change its inputs with `#10` delays, while *synthesizable* RTL must never use `#10` delays?

3 Topic 2 — Sequential logic and FSM

3.1 Background

Sequential logic has memory: outputs can depend on past inputs. State is stored in **flip-flops** that update only on a **clock edge**.

- `always_ff @(posedge clk)` describes flip-flops.
- `<=` (non-blocking) is used for clocked updates.
- A **counter** is the simplest example.

```
always_ff @(posedge clk) begin
    if (rst) count <= '0;
    else     count <= count + 1'b1;
end
```

The lab module `src/one_hz_tick.sv` counts clock cycles and, on the wrap cycle, raises `tick_1s` for exactly one clock cycle. In the AC701 implementation, `src/top.sv` passes `CLK_FREQ_HZ = 200_000_000` so the counter wraps once per second on the 200 MHz board clock.

Note

A tick is a clock *enable*, not a new clock. `tick_1s` is a normal signal that is high for one cycle. The whole design still runs on the single 200 MHz `clk`; `tick_1s` just marks the one cycle on which the FSM is allowed to advance.

A **finite state machine (FSM)** is built directly on this sequential logic: it moves through defined **states** and has three parts: a **state register** (flip-flops, *sequential*), **next-state logic** (*combinational*), and **output decode logic** (*combinational*). The lab FSM (`src/led_fsm.sv`) walks a five-state ring, lighting one of the AC701's four user LEDs at a time and then turning them all off:

```
IDLE -> MONO_LED_0 -> MONO_LED_1 -> MONO_LED_2 -> ALL_OFF -> (back to IDLE)
```

A transition happens *only when* `tick_1s` is high, which is once per second on the board. So the counter you just met is what *paces* the FSM: `tick_1s` is the clock-enable that lets the state register advance one step, without adding a second clock.

3.2 Lab 2: inspect the counter and the FSM, then simulate them together

Step 2.1 — Open the counter source.

GUI **Sources** → **Design Sources** → double-click `one_hz_tick`. Find the `always_ff` block, the register `count_value`, and the line that pulses `tick_1s`.

Step 2.2 — Open the FSM source and identify its three parts.

GUI **Sources** → **Design Sources** → double-click `led_fsm`. Locate (1) the `typedef enum` state type, (2) the state register `always_ff ...state <= next_state;`, (3) the next-state `always_comb` guarded by `if (tick_1s)` (note its first line `next_state = state;`), and (4) the output-decode `always_comb` that sets `led_fsm` from the current `state`.

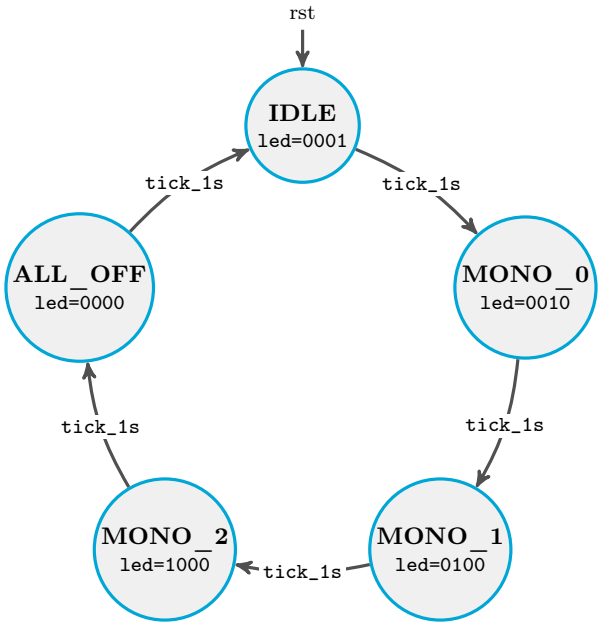

Figure 8: The `led_fsm` state diagram. Each edge fires only on the cycle when `tick_1s` is high; while `tick_1s` is low the FSM holds its current state. `led` shows `led_fsm[3:0]`, the nibble driven onto the four AC701 user LEDs.

```
8  module one_hz_tick #(
9      parameter int CLK_FREQ_HZ  = 100_000_000,
10     parameter int COUNTER_WIDTH = (CLK_FREQ_HZ <= 1) ? 1 : $clog2(CLK_FREQ_HZ)
11 ) (
12     input  logic                     clk,
13     input  logic                     rst,
14     output logic                     tick_1s,
15     output logic [COUNTER_WIDTH-1:0] count_value
16 );
17
18     localparam logic [COUNTER_WIDTH-1:0] TERMINAL_COUNT_VALUE = CLK_FREQ_HZ - 1;
19
20     always_ff @(posedge clk) begin
21         if (rst) begin
22             count_value <= '0;
23             tick_1s     <= 1'b0;
24         end else if (count_value == TERMINAL_COUNT_VALUE) begin
25             count_value <= '0;
26             tick_1s     <= 1'b1;
27         end else begin
28             count_value <= count_value + {{(COUNTER_WIDTH-1){1'b0}}, 1'b1};
29             tick_1s     <= 1'b0;
30         end
31     end
32
33 endmodule
34
```

Figure 9: `one_hz_tick.sv` in the editor.

Step 2.3 — Run the simulation once.

A single testbench, `tb_led_fsm`, exercises *both* modules together: the counter/tick and the FSM it drives. It uses a tiny `SIM_CLK_FREQ_HZ = 8` so a tick happens every 8 cycles. Run it **once** and read everything below from this one run.

GUI Make `tb_led_fsm` the sim top (right-click → **Set as Top**), then **Flow Navigator** → **Run Simulation** → **Run Behavioral Simulation**.

Note

Close the previous simulation before launching this one. If the Topic 1 waveform is still open, run `close_sim` in the Tcl Console first; otherwise Vivado may report `ERROR: [Vivado 12-1501] Simulator for snapshot ... is already running`.

PowerShell *(alternative, no GUI)*

```
vivado -mode batch -source scripts\run_simulation.tcl -tclargs tb_led_fsm
```

Tcl Console *(alternative)*

```
catch {close_sim} ;# close the Topic 1 simulation if it is still open
set_property top tb_led_fsm [get_filesets sim_1]
update_compile_order -fileset sim_1
launch_simulation
run all
```

The console prints one line per tick, ending with `tb_led_fsm completed.`

Step 2.4 — Read the counter, the tick, and the FSM in one waveform.

GUI Add `clk`, `rst`, `tick_1s`, `count_value`, `state_dbg`, and `led_fsm_out`; **Zoom Fit**. First read the **sequential** part: `count_value` increments 0 → 7 and `tick_1s` pulses for one cycle on each wrap. Then read the **FSM**: reset holds `IDLE`; after release, `state_dbg` advances one state per `tick_1s` pulse and `led_fsm_out` follows the state. The console prints one line per tick, ending with `tb_led_fsm completed.`

Checkpoint: answer in the report template

Checkpoint 2. The FSM has a state register, next-state logic, and output decode. Why does the output decode read `state` (the registered value) rather than `next_state`?

4 Topic 3 — From RTL to bitstream: synthesis and implementation

4.1 Background

So far you have simulated the lab modules on their own. To run them on a real chip they need a **top-level wrapper** that connects them to the board's physical pins. `src/top.sv` is that wrapper for the AC701:

- an `IBUFDS` buffer converts the board's 200 MHz *differential* clock (`clk_p`, `clk_n`) into the single-ended `clk` the design runs on;
- a two-flip-flop **synchroniser** cleans up the asynchronous reset button before it enters the clocked logic;

- it instantiates `one_hz_tick`, `comb_led_logic`, and `led_fsm`, passing `CLK_FREQ_HZ = 200_000_000`; and
- `sw[3]` selects which result reaches the four user LEDs: `led = sw[3] ? led_comb[3:0] : led_fsm[3:0]`.

The constraints file `constr/ac701_lab.xdc` ties each port to a physical package pin and declares the 200 MHz (5 ns) clock, so the tool knows *where* the signals go and *how fast* the clock runs.

Turning this RTL into a configured chip takes three stages:

- **Synthesis** converts the SystemVerilog into a netlist of the FPGA's building blocks (look-up tables, flip-flops, carry chains).
- **Implementation** *places* those blocks at specific sites on this exact device and *routes* the wires between them, honouring the XDC constraints.
- **Bitstream generation** writes the configuration file `top.bit` that programs the FPGA fabric.

4.2 Lab 3: build the design

Note

Do not run both methods at once. `build.tcl` deletes and recreates `build\`. If the GUI has the project open, Windows blocks the delete. **Close the GUI before running** `build.tcl`, or use the GUI buttons instead.

Option A — click through the flow.

GUI **Flow Navigator** → **Run Synthesis** (**OK**); when it finishes choose **Run Implementation** (**OK**); then **Generate Bitstream** (**OK**).

Tcl Console *(equivalent)*

```
launch_runs synth_1 -jobs 4
wait_on_run synth_1
launch_runs impl_1 -to_step write_bitstream -jobs 4
wait_on_run impl_1
```

Option B — one scripted build.

PowerShell (close the GUI first)

```
vivado -mode batch -source build.tcl
```

Watch for `INFO: Bitstream generated at: ...\impl_1\top.bit`.

Step 3.1 — Confirm it built and check for errors.

GUI Open the **Design Runs** tab; `synth_1` and `impl_1` should show green check marks. Open **Messages** and confirm no red errors.

Checkpoint: answer in the report template

Checkpoint 3. `ac701_lab.xdc` supplies two distinct kinds of information: *where* each port connects and *how fast* the clock runs. What would go wrong if the clock-period constraint were left out?

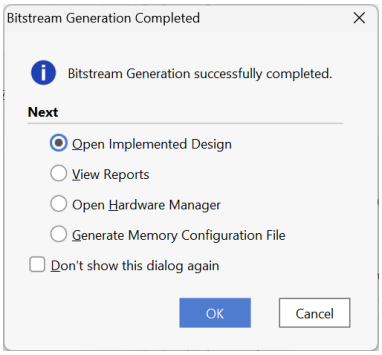

Figure 10: Design Runs after a successful build. Open the implemented design to see reports.

5 Topic 4 — Static timing analysis

5.1 Background

The AC701 board clock is 200 MHz, so the clock period is 5 ns. Every signal launched by one flip-flop on a clock edge must travel through the combinational logic, reach the next flip-flop, and *settle* before the following edge captures it. **Static timing analysis** (STA) checks this for *every* register-to-register path in the design at once. Unlike the simulations in Topics 1–2, STA uses no input vectors; it works purely from the netlist delays and the clock constraint, so it proves timing for *all* input combinations, not just the ones a testbench happens to drive.

Setup constraint (data must not arrive too late).

Start from the one-period bound and add a margin t_{unc} for **clock uncertainty** (clock jitter plus skew between the launch and capture flip-flops):

$$T_{\text{clk}} \;\geq\; t_{\text{cq}} + t_{\text{comb}} + t_{\text{su}} + t_{\text{unc}}.$$

This uses the *slowest* (longest) combinational path, so that path sets the maximum usable clock frequency. The **setup slack** is how much room is left over:

$$t_{\text{slack}} \;=\; T_{\text{clk}} - \big(t_{\text{cq}} + t_{\text{comb}} + t_{\text{su}} + t_{\text{unc}}\big).$$

Negative slack means the path is too slow to meet the clock period, and the captured value is undefined.

Hold constraint (data must not arrive too early).

Hold checks the same path in its *fastest* version, using the minimum delays $t_{\text{cq,min}}$ and $t_{\text{comb,min}}$ (setup used the slowest):

$$t_{\text{cq,min}} + t_{\text{comb,min}} \;\geq\; t_{\text{h}} + t_{\text{unc}}.$$

The corresponding **hold slack** is

$$t_{\text{slack}} \;=\; \big(t_{\text{cq,min}} + t_{\text{comb,min}}\big) - \big(t_{\text{h}} + t_{\text{unc}}\big).$$

The shortest path must be long enough that the newly launched value does not race through and overwrite the data before the capturing flip-flop has latched the *previous* value. This is a short-path problem, checked separately from setup. Because the inequality contains no T_{clk} term, a hold violation **cannot be fixed by slowing the clock**.

What Vivado reports.

A path with positive slack meets timing with room to spare; negative slack means it fails. Vivado summarises this across the whole design with:

- **WNS** (Worst Negative Slack) is the slack of the single tightest setup path. WNS ≥ 0 means *every* setup path is met.
- **TNS** (Total Negative Slack) is the sum of all negative setup slacks. TNS $= 0$ confirms there are no setup violations.
- **WHS** / **THS** are the Worst / Total Hold Slack, the hold-side equivalents of WNS / TNS.
- The **critical path** is the path with the smallest slack, i.e. the one closest to failing (and the first to fix if timing is not met).

In this small design the critical path almost always lies inside the one-second counter in `one_hz_tick`. With `CLK_FREQ_HZ = 200 000 000` the counter is about 28 bits wide, and its increment adder and terminal-count comparator form the longest combinational chain between two flip-flops; everything else (the FSM, the LED multiplexer) is far shorter.

Note

The switches and reset button are slow, human-driven inputs, so `ac701_lab.xdc` marks them `set_false_path`. STA therefore ignores paths that start at those pins and reports only the internal register-to-register paths, which is why the critical path lives inside the counter rather than at an input.

5.2 Lab 4: read the timing report and find the critical path

Step 4.1 — Open the implemented design.

GUI **Flow Navigator** → **Open Implemented Design**.

Tcl Console `open_run impl_1`

Step 4.2 — Open the timing summary.

GUI **Reports** → **Timing** → **Report Timing Summary** (**OK**).

Tcl Console `report_timing_summary`

Step 4.3 — Inspect the worst (critical) path.

GUI In the **Timing** tab expand **Setup** → your clock and click the first (worst) path. The **Path Properties** show Source (From), Destination (to), Source Clock, Destination Clock, Total Delay, Logic Delay and Net Delay. Record them.

Step 4.4 — See the path as hardware.

GUI Right-click the selected path → **Schematic**. Identify the launch flip-flop, the combinational logic between them (LUTs and the carry chain of the counter's adder/comparator), and the capture flip-flop. This chain is the physical hardware whose delay the slack in Step 4.3 measures.

Checkpoint: answer in the report template

Checkpoint 4.

1. Record **WNS** and **TNS**. Is setup timing met?
2. Record the start point and endpoint of the critical path, and describe in one or two sentences what hardware lies between them.
3. Could a *hold* violation (negative WHS) be fixed by slowing the clock?
4. (Optional) Open the timing report for the worst setup path and read off the clock period T_{clk} (the requirement), the **data path delay** ($t_{\text{cq}} + t_{\text{comb}}$), the **clock uncertainty** t_{unc}, and the **slack**. Rearrange the setup-slack equation to solve for the setup-time requirement

$$t_{\text{su}} = T_{\text{clk}} - t_{\text{unc}} - (t_{\text{cq}} + t_{\text{comb}}) - t_{\text{slack}},$$

and report your four numbers and the resulting t_{su}.

6 Topic 5 — Power analysis

6.1 Background

Vivado's power report *estimates* on-chip power and splits it into two parts: **static** power (transistor leakage, present whenever the device is powered) and **dynamic** power (drawn only when nodes switch). Vivado further breaks the dynamic part into **clock**, **logic/signal**, and **I/O** power.

Dynamic switching power follows

$$P_{\text{dyn}} \approx \alpha C V_{\text{DD}}^2 f,$$

where α is the switching (toggle) activity, C the switched capacitance, V_{DD} the supply voltage, and f the clock frequency. The activity α is the key unknown: unless you supply real switching data (e.g. a SAIF file from simulation), Vivado performs a *vectorless* estimate using assumed default toggle rates, and flags this with a low **Confidence level**. The V_{DD}^2 term is why supply voltage dominates the energy budget, and the linear f term is why the clock and clock-driven logic are often the largest dynamic contributors.

You read this report on the **AC701** implementation from Topic 3, running at 200 MHz. Because the design is tiny and its only continuously toggling logic is the free-running counter inside `one_hz_tick`, the clock network and that counter dominate the dynamic estimate, while everything else contributes very little.

6.2 Lab 5: read the power report

Step 5.1 — Open the power report.

GUI **Reports → Report Power** (**OK**).

Tcl Console `report_power`

Step 5.2 — Record the numbers.

From the **Summary**, read off the total on-chip power, the static and dynamic totals, the clock / logic-signal / I/O components, and the reported **Confidence level**; note which dynamic category is largest.

Checkpoint: answer in the report template

Checkpoint 5. Imagine halving the clock frequency to 100 MHz. Which part of the total power would drop, and which would stay roughly the same?

7 Topic 6 — Hardware targets and AUP-ZU3 connection

7.1 Background

Every report so far has come from the **AC701** project, because the university PCs ship with Artix-7 device support and can synthesize, implement, and analyse it. Your bench board, however, is the **RealDigital AUP-ZU3-4GB**, a Zynq UltraScale+ (XCZU3EG) device from a *different* FPGA family. A bitstream is device-specific, so the `top.bit` you built in Topic 3 **cannot** be programmed onto the AUP-ZU3 (and an AUP-ZU3 bitstream cannot run on the AC701).

To stay portable on PCs that may lack XCZU3EG support, the AUP-ZU3 demos are shipped as **prebuilt bitstreams** under `prebuilt_bitstreams\`. Vivado only needs its **Hardware Manager** to download a `.bit` file over JTAG, with no synthesis, implementation, or even a project required. In this topic you connect the board and then watch the *same RTL you simulated in Topics 1–2* (the combinational logic and the LED FSM) running on real hardware.

7.2 Lab 6: prepare the AUP-ZU3 for prebuilt demos

Step 6.1 — Connect and power the AUP-ZU3.

The board has **two USB-C ports**: **PROG UART** goes to the **PC** for JTAG/programming, and **EXT PWR** goes to a USB-C **PD power adapter**. The PROG UART cable alone does not power the board.

With the board **off**:

1. set the **BOOT** switch next to the microSD slot to **JTAG**, not SD;
2. connect the USB-A-to-USB-C data cable from the **PC** to **PROG UART**;
3. connect the USB-C PD adapter to **EXT PWR**;
4. power the board on and confirm the power LED turns on.

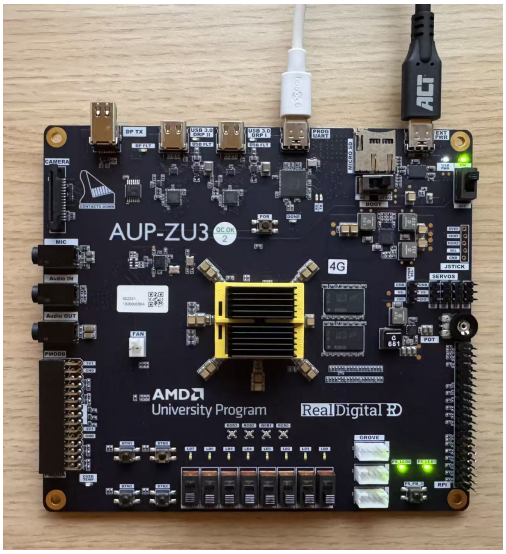

Figure 11: The powered AUP-ZU3 board with both USB-C cables attached.

Step 6.2 — Check that Hardware Manager can see the board.

Preflight before any prebuilt demo: EXT PWR is connected and the power LED is on; **BOOT** is set to **USB-JTAG/JTAG** (power-cycle after changing it); **PROG UART** is connected with a data cable; and no other Vivado, Vitis, Hardware Manager, or `hw_server` session is already using JTAG.

GUI In Vivado, **Open Hardware Manager** → **Open Target** → **Auto Connect**. The device list should contain the FPGA device, usually named `xczu3_0` or `xczu3eg`. You may also see `arm_dap_1`; do not program that entry because it is the debug access port for the ARM Processing System, not the FPGA fabric.

Tcl Console The same check from a Tcl Console is:

```
open_hw_manager
connect_hw_server
open_hw_target
get_hw_devices
close_hw_target
close_hw_manager
```

```
******** Xilinx cs_server v2023.2.0
  ****** Build date   : Oct 10 2023-06:01:56
    **** Build number : 2023.2.1696910516
      ** Copyright 2017-2022 Xilinx, Inc. All Rights Reserved.
      ** Copyright 2022-2026 Advanced Micro Devices, Inc. All Rights Reserved.


connect_hw_server: Time (s): cpu = 00:00:01 ; elapsed = 00:00:09 . Memory (MB): peak = 3643.414 ; gain = 1.332
localhost:3121
open_hw_target
INFO: [Labtoolstcl 44-466] Opening hw_target localhost:3121/xilinx_tcf/Xilinx/88022500036AA
get_hw_devices
xczu3_0 arm_dap_1
close_hw_target
INFO: [Labtoolstcl 44-464] Closing hw_target localhost:3121/xilinx_tcf/Xilinx/88022500036AA
close_hw_manager
```

Figure 12: The AUP-ZU3 detected in Hardware Manager.

Note

Use only one JTAG connection at a time. Close Hardware Manager before running a programming script, or let the script open Hardware Manager itself. If a script says the target is busy, close other Vivado or Vitis windows and retry.

Step 6.3 — Program the LED demo and watch the lab RTL run on hardware.

With the board detected, program the prebuilt **LED demo**: the AUP-ZU3 build of the very same `one_hz_tick`, `comb_led_logic`, and `led_fsm` modules you read and simulated in Topics 1–2, wrapped for this board by `src/top_aup_zu3.sv`.

GUI In **Hardware Manager** right-click the `xczu3` device → **Program Device**, set the bitstream to `prebuilt_bitstreams\aup_zu3_led_demo.bit`, and click **Program**.

PowerShell *(alternative)* from the repository root (the script opens Hardware Manager itself, so close any open one first):

```
vivado -mode batch -source scripts\program_prebuilt_led_jtag.tcl
```

Once programmed, `sw[7]` chooses the mode:

- `sw[7]=0` (**sequential FSM demo**): the lower LEDs advance one state per second, `led[0]` (`IDLE`) → `led[1]` → `led[2]` → `led[3]` → all off, then repeat. Press the reset button to jump straight back to `IDLE`.
- `sw[7]=1` (**combinational demo**): `led[7:0]` show the eight logic functions of `sw[3:0]` *live*. Flip `sw[3:0]` and the LEDs change immediately, because there is no clock.

Troubleshooting

If **Auto Connect** finds nothing, or a prebuilt demo script reports that no FPGA device was found:

- **Power.** Is **EXT PWR** plugged in and the power LED on? `PROG UART` alone does not power the board.
- **Boot mode.** Is the **BOOT** switch on **JTAG** (not SD)? Power-cycle after changing it.
- **JTAG busy.** Close any other Vivado/Vitis or running script holding the target.
- **Wrong target.** Program the `xczu3`/`xczu3eg` FPGA device, not `arm_dap`.

8 Topic 7 — PWM DAC: the digital/analog boundary

8.1 Background

A digital output pin can only be 0 or 1, yet we often want an *analog* level, for example a dimmable LED. **Pulse-Width Modulation (PWM)** fakes one by switching the pin on and off very fast and varying the **fraction** of each period that it stays high. That fraction is the **duty cycle**. The LED (and your eye) average the fast square wave, so the perceived brightness follows the duty cycle.

PWM is built from two pieces you have already met: a **sequential** counter and a **combinational** comparator. A free-running N-bit counter ramps $0, 1, 2, \ldots, 2^N-1$ and wraps; on every clock the output is

$$\texttt{pwm\_out} = (\texttt{count} < \texttt{duty}).$$

So `pwm_out` is high for the first `duty` counts of each 2^N-cycle period and low for the rest, and its average value, and therefore the LED brightness, is $\texttt{duty}/2^N$. The waveform below shows how the comparator turns the counter ramp into a pulse train:

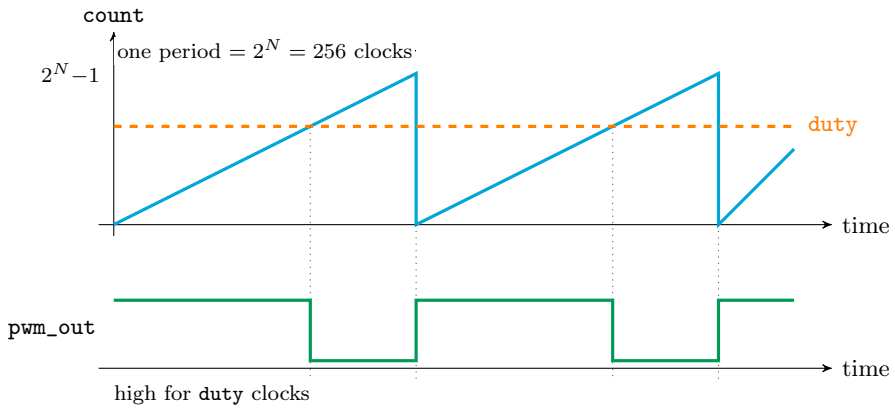

Figure 13: PWM from a counter and a comparator. While the counter ramp is below `duty`, `pwm_out` is high; the high fraction, and thus the average level that drives the LED, equals $\texttt{duty}/2^N$. Raising `duty` lifts the dashed line and widens every pulse.

- With $N = 8$ the period is $2^8 = 256$ clocks. At the AUP-ZU3's 100 MHz clock the PWM frequency is 100 MHz/256 = 390.625 kHz, far above what the eye can follow, so you see only the average.
- `duty_select` turns the four switches into a duty code, `duty = {sw, 4'b0000}`: 16 evenly spaced levels from `0000_0000` (0%) up to `1111_0000` = 240/256 ≈ 93.75%. This simple selector never reaches a true 100%.

The full data path strings these together:

8.2 Lab 7: complete the PWM generator and watch it dim an LED

You will **create the project, complete two lines of RTL, and run the behavioural simulation**. You do *not* synthesize or implement anything; the board demonstration in Step 7.4 uses a prebuilt AUP-ZU3 bitstream, so a passing simulation is all your own code needs.

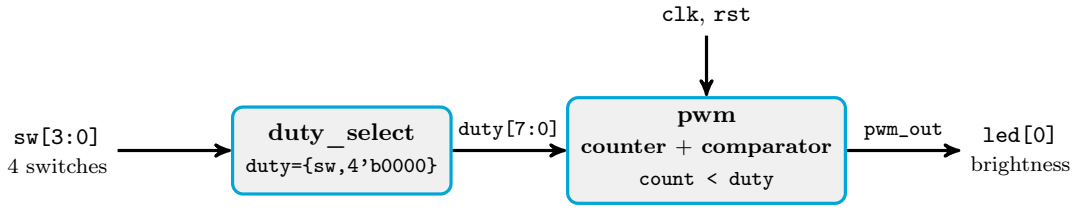

Figure 14: The PWM data path. The four switches choose a duty code; the counter-plus-comparator turns it into a pulse train whose average dims `led[0]`. (`clk` comes from the board's `IBUFDS`; `rst` from the reset synchroniser.)

Step 7.1 — Create the PWM project (no synthesis).

PowerShell from the repository root. The first line builds the project (fast, no synthesis), and the second opens it in the GUI:

```
vivado -mode batch -source pwm_dac\scripts\create_pwm_project.tcl
vivado pwm_dac\build_pwm_project\pwm_dac_lab.xpr
```

Step 7.2 — Inspect the design and complete `pwm.sv`.

GUI In **Sources** → **Design Sources**, open and read the four small modules so you see the whole data path from the figures above:

- `duty_select.sv` is *combinational*: it turns `sw[3:0]` into the 8-bit `duty`.
- `pwm.sv` holds the *sequential* counter and *combinational* comparator you will finish.
- `synchronizer.sv` cleans the asynchronous reset button into the clock domain.
- `top_pwm_dac.sv` is the board wiring: the `IBUFDS` clock, the modules above, and the LED map.

Now open `pwm.sv` and fill in its two TODO lines so the code matches the waveform:

1. **TODO 1: the counter (sequential).** Inside the `always_ff`, make `count` advance by one each clock and let the N-bit register *wrap* on its own at the top of the range. This is the rising ramp in the figure.
2. **TODO 2: the comparator (combinational).** Drive `pwm_out` high exactly while the counter sits below the threshold, that is, compare `count` against `duty`. This is the crossing point in the figure.

Step 7.3 — Run the behavioural simulation and confirm it passes.

`tb_pwm` drives several duty values and counts how many cycles `pwm_out` is high in each 256-cycle period, checking it equals `duty`.

GUI `tb_pwm` is already the simulation top, so **Flow Navigator** → **Run Simulation** → **Run Behavioral Simulation**.

Note

Don't see the `PWM_STUDENT_TEST_PASS` line? The GUI runs only a short default time (e.g. 1 μs) and then pauses. Click **Run All** on the simulation toolbar (or type `run all` in the Tcl Console) so the simulation continues to `$finish` and prints the result.

PowerShell *(alternative, no GUI)*

```
vivado -mode batch -source pwm_dac\scripts\run_pwm_simulation.tcl
```

A correct implementation ends with a clear pass line in the Tcl console (and in `pwm_dac/build_pwm_dac/xsim_pwm/xsim.log`):

```
PWM_STUDENT_TEST_PASS: all 7 duty-cycle checks passed.
```

If you see a `[FAIL]` line instead, compare its `high_cycles` with `expected` for that duty, fix `pwm.sv`, and rerun.

Step 7.4 — See it dim an LED on the board (prebuilt demo).

Because the campus PCs may not implement an XCZU3EG design, the board demonstration uses a prebuilt AUP-ZU3 bitstream, with no synthesis or implementation on your part. (It shows the expected brightness; your *required* proof remains the passing testbench from Step 7.3.) After the AUP-ZU3 connection preflight in Topic 6:

GUI In **Hardware Manager** right-click the `xczu3` device → **Program Device**, choose `prebuilt_bitstreams\aup_zu3_pwm_led_demo.bit`, and click **Program**.

PowerShell *(alternative)*

```
vivado -mode batch -source pwm_dac\scripts\program_prebuilt_pwm_jtag.tcl
```

Set `sw[3:0]` low (e.g. `0001`) and `led[0]` glows dimly; raise toward `1111` and it brightens. `led[4:1]` mirrors your switch code so you can read off the selected level.

Checkpoint: answer in the report template

Checkpoint 7. Put a screenshot of the `PWM_STUDENT_TEST_PASS` line from your simulation log in the report.

9 Topic 8 — Optional rainbow RTL demo

9.1 Just for fun

If your group finishes early, you can program a prebuilt rainbow demo for the AUP-ZU3. It drives the four RGB LEDs through a slowly changing hue generator. The switches still mirror onto the single-colour LEDs, and the pushbuttons change the rainbow speed, direction, and phase spacing.

Step 8.1 — Program the prebuilt rainbow demo.

Keep the same AUP-ZU3 connection and JTAG preflight from Topic 6, then program the prebuilt rainbow bitstream:

```
vivado -mode batch -source rainbow\scripts\program_prebuilt_rainbow_jtag.tcl
```

Step 8.2 — Play with the controls.

- `sw[7:0]` mirror directly onto `led[7:0]`.
- Button 0 makes the colour cycle faster.
- Button 1 makes the colour cycle slower.
- Button 2 reverses the colour direction.
- Button 3 changes the phase spacing between RGB LEDs.

Lab completion checklist

Note

- **Board return is required.** Your group can only pass the lab after returning the FPGA board box to the responsible TA..
- **Report submission.** Write up your checkpoint answers in the provided **LaTeX report template** (`EE2C2_Lab3_Report_Template`) available on **Brightspace**, and submit the compiled report PDF there. This report is the deliverable that is graded.